

Fine-tuning, Reinforcement Learning and Parameter-Efficient Adaptation

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center

August 28, 2026

Part I

Finetuning

Outline

1 Naively Finetuning LLMs

- What is finetuning?
- Why not (fully) finetune LLMs?

2 Making finetuning practical through LoRA

- Understanding why LoRA works
- How to use LoRA

3 Making finetuning scalable through QLoRA

- Quantizing LLMs: Why and how?
- How does it differ from LoRA?

Section 1

Naively Finetuning LLMs

Subsection 1

What is finetuning?

Why finetune LLMs?

Prompting vs finetuning

- **Prompting:** Changes **input behavior**. We guide the model with instructions and examples.
- **Fine-Tuning:** Changes **model parameters**. We switch the **knowledge** of the model.

Reminder

Fine-Tuning (FT): The process of further training a pre-trained model on a smaller, domain-specific dataset to adapt its general knowledge to a specialized task.

Pretraining vs Finetuning

Reminder

- **Pre-training** provides the model with initial, **general** knowledge by utilizing **any** and all data sources.
- **Finetuning** shifts existing knowledge, to **specific** domains using **domain-specific** datasets.

Example: Domain-specific tasks

Part-of-Speech (POS) Tagging

The process of assigning grammatical categories to each word in a sentence.

POS Tagging

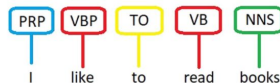


Figure: <https://byteiota.com/pos-tagging/>

Example: Domain-specific tasks

Biomedical POS Tagging

- Patterns absent or rare in general-domain text.
- Prompting probably not enough due to **missing knowledge**.

Word	POS Tag
Homozygous	JJ
null	JJ
mutants	NNS
die	VBP
shortly	IN
after	RB
birth	NN

Table: Example from the CRAFT dataset.

Section 1

Naively Finetuning LLMs

Subsection 2

Why not (fully) finetune LLMs?

Full finetuning

Power and Storage

- **Computational Power:** Full FT requires **massive hardware requirements** and **thousands of compute hours** for large models (e.g., Llama-70B).
- **Storage:** Each specialized task requires a separate, full copy of the massive foundational model.

Full finetuning

Training and deployment

- Training is **slow**.
- Deployment becomes complex, managing **multiple large models** for different applications.
 - A single 13B model may require 26GB of storage.
 - 10 finetuned models may thus require 260GB.
- High risk of **Catastrophic Forgetting**—the model forgets its general knowledge while learning the specific task.

Scalability issues

BERT

- **Model Size:** ~340M Parameters
- **VRAM Requirement:** ≈ 8-12GB
- **Training Time:** Minutes to a few hours

Llama 70B

- **Model Size:** ~70 Billion Parameters
- **VRAM Requirement:** ≈1000GB+
- **Training Time:** Days to Weeks (even with specialized hardware)

LLMs are **too large** to run full-finetuning. But do we need to?

Parameter-Efficient Fine-Tuning (PEFT)

Definition (PEFT)

Parameter-Efficient Fine-Tuning: A set of techniques used to adapt large pre-trained LLMs to specific tasks by updating only a **small subset of their parameters**, rather than retraining the entire model.

How many such methods are used?

- Many algorithms developed through the years (adapters, prompt/prefix tuning etc.).
- However, the **LoRA** family of methods has dominated the LLM space in the last few years.

Summing up

Why efficient finetuning is needed

- We need finetuning to provide new knowledge to the model.
- However, further pretraining the model is prohibitively expensive.
- There are optimized methods for finetuning using a fraction of compute called PEFT.

Outline

- 1 Naively Finetuning LLMs
 - What is finetuning?
 - Why not (fully) finetune LLMs?
- 2 Making finetuning practical through LoRA
 - Understanding why LoRA works
 - How to use LoRA
- 3 Making finetuning scalable through QLoRA
 - Quantizing LLMs: Why and how?
 - How does it differ from LoRA?

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Section 2

Making finetuning practical through LoRA

Subsection 1

Understanding why LoRA works

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

PEFT: LoRA

Definition (LoRA)

Low-Rank Adaptation is a specific PEFT algorithm that exploits the fact that weight updates in LLMs have a “low intrinsic rank”.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Low-rank matrices

What is a matrix?

A matrix is essentially a organized collection of data (rows and columns). For large datasets, these matrices can become incredibly complex. However, low rank matrices enable us to **scalably** represent information.

Example: Data matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix}$$

Low-rank matrices

Matrix rank

- Rank r represents the number of genuinely independent columns (or rows!) within the matrix M .
- If $M \in \mathbb{R}^{m \times n}$, then $\text{rank}(M) \leq \min(m, n)$.

Example: Music

- Imagine a piano with 88 keys.
- Most songs don't require all 88 keys; they can be described by melody, rhythm etc.
- Low-rank updates capture changing **general patterns**.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Low-rank matrices

Rank Factorization

If a matrix M has a low rank r , we can approximate it (or exactly represent it) by factoring it into a product of two simpler matrices.

Example: Rank 1 matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}.$$

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Why are LLM updates typically low-rank?

Many possible explanations

- Updates generally follow the direction of existing knowledge → the updates may often just be **re-weighting** of existing weights.
- Underlying **ML problems** are usually inherently low-rank.

Why are LLM updates typically low-rank?

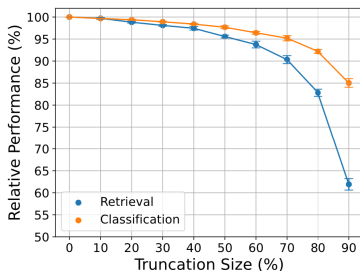


Figure: <https://aclanthology.org/2025.emnlp-main.1410.pdf>

Large embedding spaces

Modern NLP models are **over-parameterized** → not enough information to fill in higher ranks of weights.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

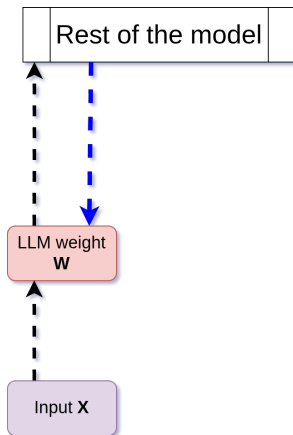
LoRA: Low-rank decomposition

$$\underbrace{A_{m \times n}}_{\text{large matrix}} \approx \underbrace{U_{m \times r}}_{\text{left factors}} \underbrace{V_{r \times n}}_{\text{right factors}}, \quad r \ll \min(m, n).$$

$$A_{10000 \times 5000} \approx U_{10000 \times 50} V_{50 \times 5000},$$

$$\underbrace{10000 \times 5000}_{50,000,000 \text{ parameters}} \longrightarrow \underbrace{10000 \times 50 + 50 \times 5000}_{750,000 \text{ parameters}}.$$

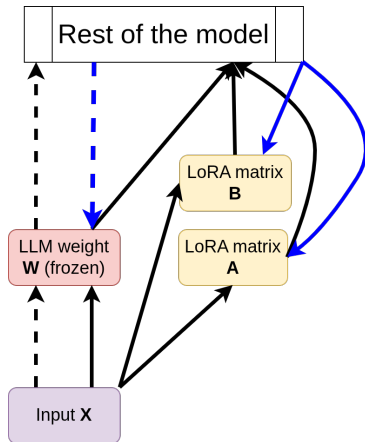
LoRA: Mathematical Foundation



Full SFT

- $W \in \mathbb{R}^{d \times k}$ weights.
- X input, h output.
- Fwd pass: $h = WX$.
- Bckwd pass:
 $W = W + \alpha \nabla h$

LoRA: Mathematical Foundation



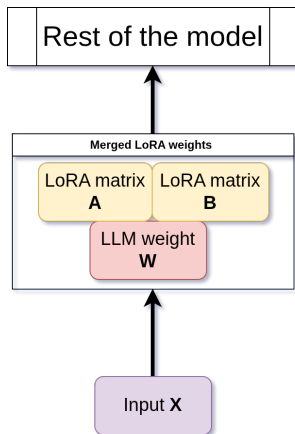
LoRA

- $A \in \mathbb{R}^{r \times k} \rightarrow \text{Gaussian.}$
- $B \in \mathbb{R}^{d \times r} \rightarrow \text{zeros.}$
- $r \ll \min(d, k)$, hence A, B are much smaller than W .

Example: Music

A could represent the genre of music, while B the melody, rhythm etc.

LoRA: Mathematical Foundation



LoRA

- During inference, we compute
$$H = WX + \Delta WX.$$
- Alternatively, merge weights:
$$W' = W + \frac{\alpha}{r}(AB)$$

LoRA: Algorithm

▷ =====*Initialization*===== ◁

for all $W \in all_weights$ **do**

└ Set $\mathbf{W}$ to read-only ▷ *Original knowledge preserved*

for all $W_u \in target_modules$ **do**

└ Initialize $\mathbf{A}$ randomly

└ Initialize $\mathbf{B} = \mathbf{0}$ ▷ *Contributes zero to the output initially*

▷ =====*Backpropagation*===== ◁

for all $X \in input_embeds$ **do**

└ $\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$

└ $\mathbf{H} = \mathbf{W} \mathbf{X} + \Delta \mathbf{W} \mathbf{X}$ ▷ *Output = original + adaptation*

▷ =====*End of training*===== ◁

$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r} (\mathbf{A} \mathbf{B})$ ▷ *Merge original and adaptation*

Section 2

Making finetuning practical through LoRA

Subsection 2

How to use LoRA

LoRA: Practical Considerations

How many ranks do I use?

- $r \in [1, 16]$: fast, easy, recommended for minor adjustments.
- $r \in [16, 64]$: recommended for most domain adaptation scenarios.
- $r \in [64, 256]$: may hit computational bottlenecks, only use for dramatic shifts in domain and knowledge.

LoRA: Practical Considerations

How do I set α ?

- Usually set to a multiple of r .
- Very low $\alpha \rightarrow$ minimal contribution. Very high $\alpha \rightarrow$ destruction of model knowledge.
- $\alpha = 2r$ usually used as a starting point.

LoRA: Practical Considerations

What modules do I change?

Start from the top of this list and go down as required:

- 1 **Query (q_proj) and Value (v_proj):** Most frequently adapted; control **how the model attends** to input.
- 2 **Key (k_proj):** Increases **information retrieval** capacity.
- 3 **Output (o_proj):** Less common, but useful for modifying **integrated** attended information.
- 4 **Feed-forward Networks (MLP):** Possible to adapt, but generally offer less parameter benefit compared to attention layers.

LoRA: Practical Considerations

Should we merge the LoRA weights?

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}(\mathbf{AB})$$

Yes

- Lowers **GPU memory** and execution **time**.
 - More significant if r is large.
- Deployment may be easier.

No

- You can use the same model with **many** finetuned adapters which are **dynamically selected** depending on your task.

Summing up

LoRA

- Weight updates in LLMs are generally low-rank.
- LoRA exploits this fact to create new, much smaller weights, updated through finetuning.
- We can adjust the rank of these matrices to tune how much finetuning is desirable, although compute increases significantly for high ranks.
- Merging weights may be desirable in some specific cases.

Outline

- 1 Naively Finetuning LLMs
 - What is finetuning?
 - Why not (fully) finetune LLMs?
- 2 Making finetuning practical through LoRA
 - Understanding why LoRA works
 - How to use LoRA
- 3 Making finetuning scalable through QLoRA
 - Quantizing LLMs: Why and how?
 - How does it differ from LoRA?

- └ Making finetuning scalable through QLoRA
- └ Quantizing LLMs: Why and how?

Section 3

Making finetuning scalable through QLoRA

Subsection 1

Quantizing LLMs: Why and how?

QLoRA: Quantization

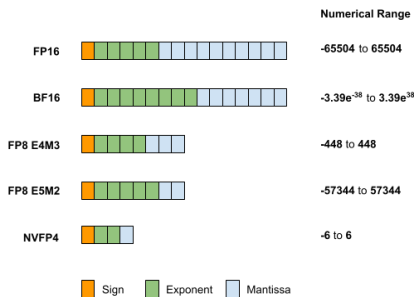


Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

What is it?

- Quantization refers to dropping the size (not number) of the LLM parameters.
- Normally, each parameter is stored in a 32 bit floating point.
- Quantization can drop them to usually 4 bits, resulting in **×8 less** GPU memory and inference costs.

└ Making finetuning scalable through QLoRA

└ Quantizing LLMs: Why and how?

QLoRA: Quantization

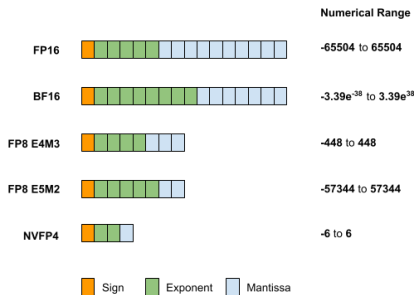


Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

Why is it useful?

- Quantization is one of the **most useful tools** at your disposal.
- Allows for training and inference in consumer-grade hardware.
- We will return to this in Module 5.

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

Section 3

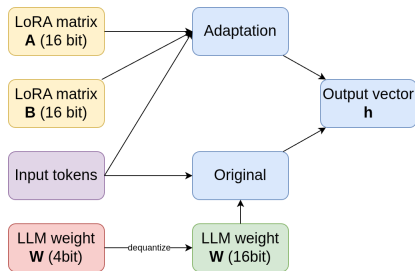
Making finetuning scalable through QLoRA

Subsection 2

How does it differ from LoRA?

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

QLoRA: Concept



- Load a 4-bit quantized model using a normal approximation of weights.
- Run LoRA in full precision, **dequantizing** the model weights **one at a time**.
- Continue as normal.

- └ Making finetuning scalable through QLoRA

- └ How does it differ from LoRA?

QLoRA: Algorithm

▷ =====Initialization=====

▷ Same as before

▷ =====Backpropagation=====

for all $X \in \text{input_embeds}$ **do**

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$$

$\mathbf{H} = \text{dequant}(\mathbf{W}) X + \Delta \mathbf{W} X$ ▷ Output = original + adaptation

▷ =====End of training=====

▷ Only attempt dynamic merge. Ignore this advice at your own peril, or if you really know what you are doing.

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

QLoRA: Pros and cons

Pros

- Frozen weights no longer monopolize **GPU memory** and inference **time**.
- Allows training in **consumer-grade** hardware.
- Massively **reduces training** times.

Cons

- End-result is a quantized → **less powerful** model.
- Dequantization imposes a $\approx 10\%$ tax on training times.
- Merging is now much messier.

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

Summing up

QLoRA

- Quantization is a mathematical methodology that reduces the size of LLM parameters.
- QLoRA uses quantization on the LLM weights while retaining the original LoRA adapter weights.
- Due to quantization, smaller models can be finetuned even on commercial hardware.

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

What did we learn?

Finetuning

- Full finetuning for LLMs is out of the question.
- However, weight updates are usually **low-rank**.
- Thus, we can **approximate** full finetuning with LoRA.
- We may use QLoRA instead to make it much more **scalable**.

Part II

Reinforcement Learning

Outline

4 General RL

- Fundamentals
- Core Concepts

5 LLM RL

- LLM RL vs general RL
- Finetuning vs RL

Section 4

General RL

Subsection 1

Fundamentals

What is RL?

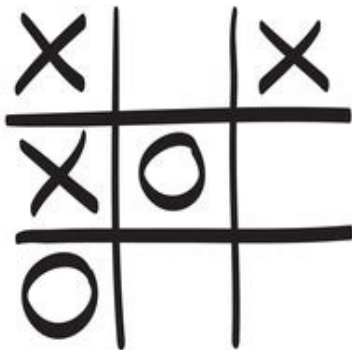
Definition (Reinforcement Learning)

A machine learning paradigm where an Agent **learns** optimal behavior through **interaction**. The system learns autonomously by trial and error, maximizing a cumulative **Reward Signal**.

Example

A master chess player makes a move. The choice is informed both anticipating possible replies and counterreplies and by immediate, intuitive judgments of the desirability of particular positions and moves. (Sutton & Bartol: "RL An Introduction", 2nd Ed).

Example: Tic-tac-toe



How would you train an agent?

- Minimax: Thrown off due to suboptimal play.
- Dynamic prog.: Needs a perfect configuration of the opponent.
- A human would learn by doing (and failing).

Example: Tic-tac-toe

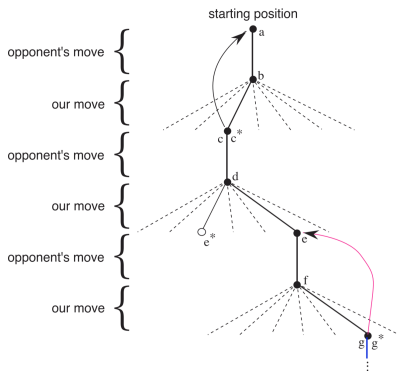


Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

Running simulations

- Set up a table where each cell = one possible ttt configuration.
- All states where no players win are given a value of 0.5 (we do not know who would win).
- All states where we win are 1, otherwise 0.

Example: Tic-tac-toe

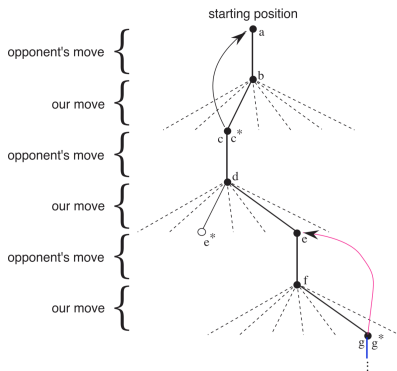


Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

Running simulations

- At each step, consider possible moves.
- Take the best one (exploitation), or a random one (exploration).
- Update previous steps with results. $V[s] \leftarrow V[s] + \alpha[V[s'] - V[s]]$

Other methods

- In reality, we use much more sophisticated and optimized methods.
- Most nonetheless follow the paradigm of $NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate]$.
- The DPO and GRPO methods used in modern-day LLM training also follow it.
- We obtain those estimates from the **environment**.

Section 4

General RL

Subsection 2

Core Concepts

General method

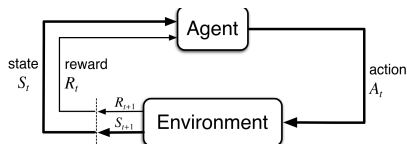


Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

The learning loop

At each step (at state S_t):

- Take action A_t .
- Move from state $S_t \rightarrow S_{t+1}$ and observe reward R_{t+1} .
- The **environment** is responsible for providing S_{t+1} and R_{t+1} given A_t .

The Environment

s	s'	a	$p(s' s, a)$	$r(s, a, s')$
high	high	search	α	r_{search}
high	low	search	$1 - \alpha$	r_{search}
low	high	search	$1 - \beta$	-3
low	low	search	β	r_{search}
high	high	wait	1	r_{wait}
high	low	wait	0	r_{wait}
low	high	wait	0	r_{wait}
low	low	wait	1	r_{wait}
low	high	recharge	1	0
low	low	recharge	0	$0.$

Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

What is the environment?

All interactions which the agent must **not** know about.

Example: Poker

The hands of other players.

What the environment isn't

If the agent is a paramedic, it should not immediately know the internal injuries of a victim.

The Environment

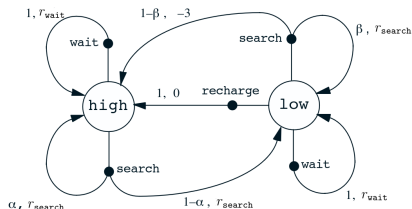


Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

What is the environment?

All interactions which the agent must **not** know about.

Example: Poker

The hands of other players.

What the environment isn't

If the agent is a paramedic, it should not immediately know the internal injuries of a victim.

The Reward

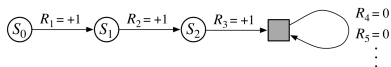


Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

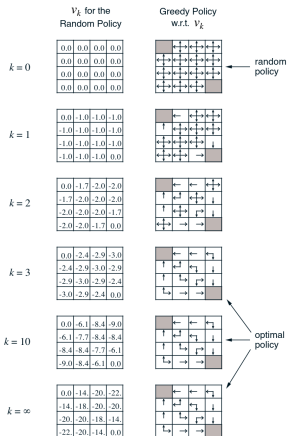
Definition (Reward r_t)

A numerical value representing degrees of success given at time t .

Definition (Return G_t)

- Discounted returns over t steps.
- $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.
- As $\gamma \rightarrow 1$ the agent prioritizes future rewards.

Policies and state-value-actions

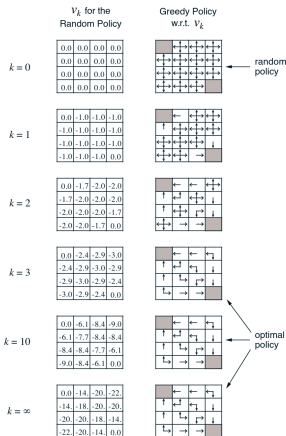


Definition (Policy π)

The agent's strategy, which dictates the behavior by specifying the probability of taking every possible action when the agent finds itself in a specific state.

Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

Policies and state-value-actions

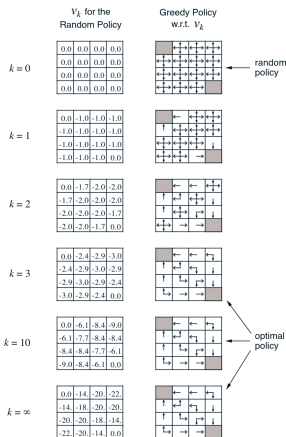


Definition (Value State Function V)

Estimates the expected future discounted reward an agent will accumulate starting from a particular state, assuming it follows a specific policy.

Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

Policies and state-value-actions



Definition (Value-Action State Function Q)

Estimates the expected future discounted reward an agent will accumulate starting from a particular state and immediately taking a specific action.

Figure: Sutton & Bartol: "RL An Introduction", 2nd Ed

Generalizing the RL Loop

- 1: Initialize policy π and value/Q-function V/Q
- 2: Initialize Experience Buffer $\mathcal{D} \leftarrow \emptyset$
- 3: **while** D not satisfied (e.g., episodes remaining) **do**
- 4: ▷ *Interaction / Data Collection* ◁
- 5: **while** s not terminal **do**
- 6: Select action $\alpha \leftarrow \pi(s)$
- 7: $r, new_s \leftarrow environment(s, \alpha)$
- 8: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, \alpha, r, new_s)\}$
- 9: $s \leftarrow new_s$
- 10: ▷ *Learning / Parameter Update* ◁
- 11: Sample a batch of experiences $\mathcal{B}$ from $\mathcal{D}$
- 12: UpdateParameters($\mathcal{B}$, Algorithm Specific Rules)
- 13: DiscountRewardEstimate $\leftarrow$ Calculate($\mathcal{B}$)

Outline

4 General RL

- Fundamentals
- Core Concepts

5 LLM RL

- LLM RL vs general RL
- Finetuning vs RL

Section 5

LLM RL

Subsection 1

LLM RL vs general RL

RL for LLMs

When training LLMs

We learn a policy $\pi(s|\alpha)$ where:

- α is an action (here next chunk of text).
- s is the state (partial generation process for next token).

Instead of explicit V or Q functions we use an **Advantage function** as a **gradient**, which estimates how better an action is given the current state (discounted at time-step t in the future).

Reward modeling

Pairwise Preference

- Response a is better than response b
- Loss: Bradley-Terry Model (Pairwise)

Rankings

- Response
 $a > b > c > \dots$
- Loss: Plackett-Luce Model1 (Rankings)

Reward modeling types

- 1 Explicit Reward Modeling.
- 2 Implicit Reward Modeling
- 3 Outcome Reward Modeling (ORM)
- 4 Process Reward Modeling (PRM)
- 5 Iterative RL with Adaptive Reward Models

Reward modeling types

Explicit Reward Modeling:

- Directly defines the reward function.
- Human annotations, automated metrics, heuristics, A.I. modules...
- Pros: Precise reward estimates.
- Cons: Time consuming, expensive.

Reward modeling types

Implicit Reward Modeling:

- Observes secondary indicators from which it infers rewards.
- e.g., Comment quality by upvote count or click-through rate.
- Pros: Can easily build vast training datasets.
- Cons: The model may learn how to “cheat” by maximizing the indicators, instead of the actual goal.

Reward modeling types

Outcome Reward Modeling (ORM):

- Just measures the end result.
- Example: Our Tic-tac-toe example!
- Pros: As straightforward as it gets.
- Cons: Credit assignment problem: The model does not know why its responses work.

Reward modeling types

Process Reward Modeling (PRM):

- Like ORM but assigns reward at in-between steps.
- Improves step-by-step reasoning: helps in many domains (math, legal, code), transparent
- Pros: Precise reward estimates.
- Cons: Difficult annotation process: Limited datasets

Reward modeling types

Iterative RL with Adaptive Reward Models:

- Reward function is dynamic.
- Bypasses reward hacking and reward model drift.
- More on that later!

Policy Optimization

- Given a trained reward model, learns relative preferences.
- Learn a policy $\pi(\alpha|s)$ where:
 - α are actions (here next tokens).
 - s is the state (partial generation process for next token)
- We can use the **Advantage** function as a gradient, which estimates how better an action is given the current state, discounted at time-step t in the future.

Section 5

LLM RL

Subsection 2

Finetuning vs RL

Finetuning

Pros

- Very easy to implement —continuation of pretraining phase.
- Techniques such as LoRA alleviate drawbacks.
- Model learns to comply with task-specific guidelines, requirements, formats.

Cons

- Computationally heavy.
- Relies on high-quality, human-curated/created data.
- Overfitting.

Reinforcement Learning

Pros

- Not as computationally heavy as SFT.
- Factual accuracy.
- Adaptability.

Cons

- “State” is evolving text sequence.
- Feedback is sparse, delayed and subjective.
- Multiple conflicting objectives.

RL has greater potential, but requires careful planning, since LLMs are **not ideal targets** for RL.

Summing up

Reinforcement Learning

- RL allows an agent to learn by themselves by interacting with their environment.
- The agent takes actions permitted by the environment and receives feedback.
- The feedback is used to calculate values for each possible state (or state-action pair).
- The agent iteratively learns using the updated value estimates.
- While the underlying learning mechanism is similar for LLMs RL may not be as effective as finetuning due to their nature.